PYTHON POLYMORPHISM

POLYMORPHISM

- Poly means "many" and morphs means "forms". So, polymorphism means many forms. (Having many forms)
- If a same method / single method performs the **different operations with same / different signatures**, then it is called as polymorphism.
- Signature ← (Number of arguments, Type of arguments)
- One name many forms
- It is one of the core object oriented programming technique.

Usage

- It is mainly used to implement the **concept of inheritance** in oops.

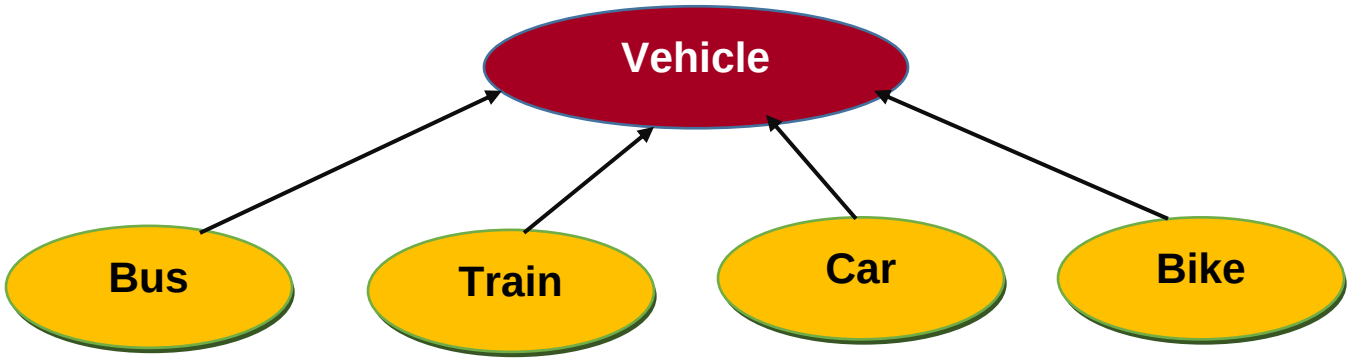


Figure 1. Polymorphism

TYPES OF POLYMORPHISM

- In python, polymorphism is classified as two types. They are
  1. Compile Time Polymorphism
  2. Run Time Polymorphism

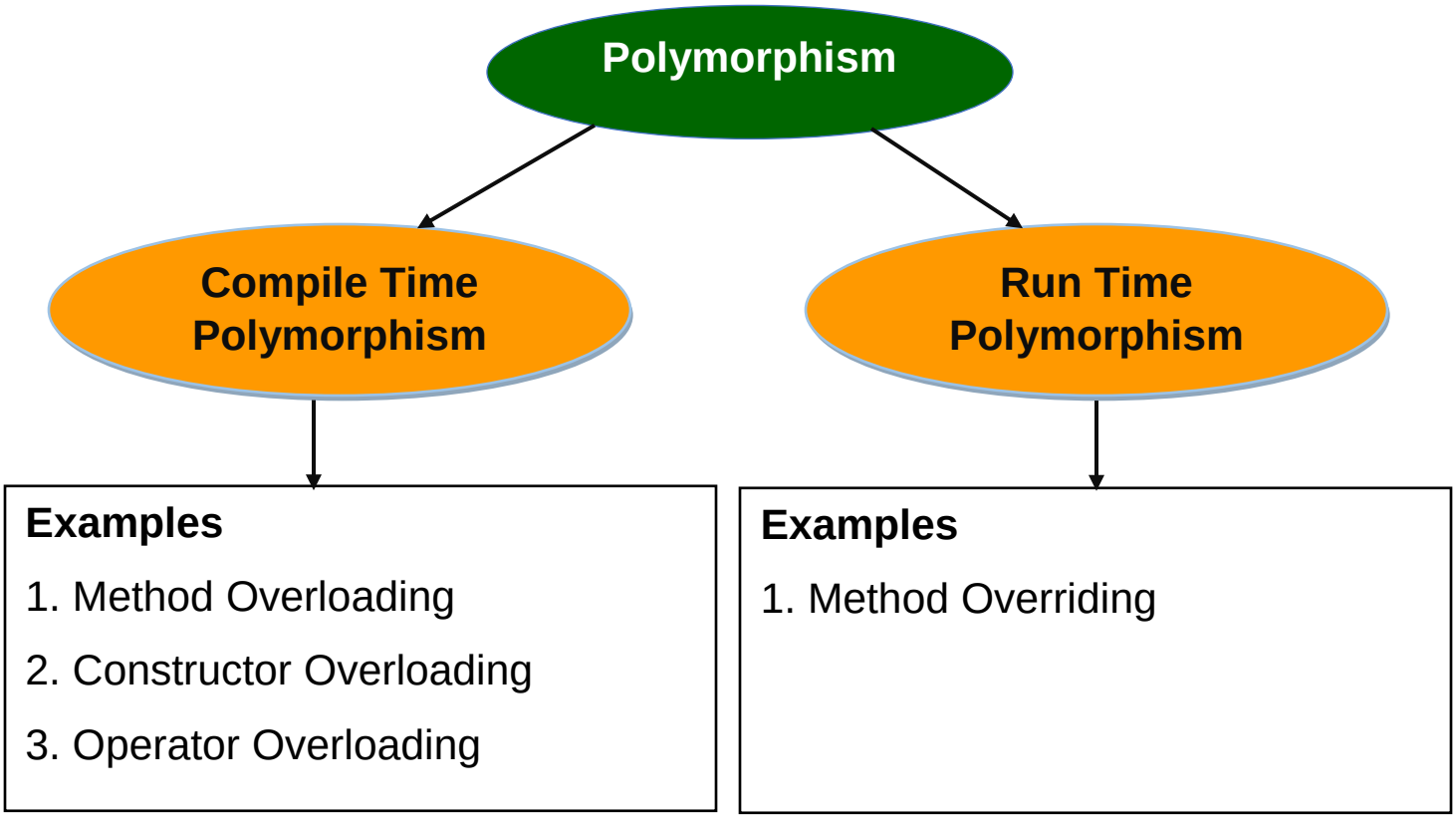


Figure: Types of Polymorphism

I. COMPILE TIME POLYMORPHISM

- Same function (single function) performs different operations using **different signatures**
- Signatures ← Number of arguments, Type of arguments
- This is called using its signatures
- Ex.
  1. Method overloading
  2. Constructor overloading.

NOTE

- Python does **not support constructor overloading**.

Function Overloading (Method Overloading)

- If same method / single method takes one or more number of arguments that is called as method overloading
- Same method (single method) performs different operations using different signatures
- This is called by using its signatures
  - Types of arguments → (int, float, void, char)
  - Number of arguments → (0,1,2,3, … n)
- **Types**
  1. Argument based function overloading
  2. Type based function overloading

Note

- Python does not support type based method overloading as it supports dynamic typed system

1. Argument based function overloading

- Same function performs different operations using different sets of arguments (0, 1, 2, etc, …) that is called as argument based function overloading
- It gives the importance to arguments (not types)
- This is called by using its signatures. Here the signature depends on the arguments of functions

Calling

- Calling is purely based on arguments order. It does not provide the priority to data types

DEFAULT ARGUMENTS BASED METHOD OVERLOADING

- Python does not directly support function overloading. Since the method arguments do not have a data type, it does not support overloading
- But it indirectly supports function overloading using default arguments
- Unlike c++/java/c#, **defining multiple methods with same name are not possible to achieve function overloading in python**
- In python, **function overloading is done by using default arguments**. Here, same function is only allowed for defining different operations instead of using multiple methods

1. DEFAULT ARGUMENTS BASED METHOD OVERLOADING

(overloading.py)

I. SOURCE CODE

Function Overloading – Only single method with default arguments are allowed.

```
# class definition
from numpy import double
class Moverloading:
    def area(self, x, y=None):
        if(x!=None and y!=None):
            return(x*y)
        elif (y==None and type(x)==double):
            return(3.14*x*x)
        else:
            return (x*x)
# object creation for current class
obj=Moverloading()
print("------------------------------")
print("\t Method Overloading")
print("------------------------------")
```

```
# calling methods based on the signatures
square=obj.area(5)
rect=obj.area(100,20)
circle=obj.area(9.0)
print("Area of the Square: ",square)
print("Area of the Rectangle: ",rect)
print("Area of the Circle",circle)
```

II. OUTPUT

```
TERMINAL    JUPYTER    PROBLEMS    OUTPUT    DEBUG CONSOLE

PS C:\Users\Ganesh\Documents\VSC Python Projects\poly> py overloading.py
-------------------------------------
        Method Overloading
-------------------------------------
Area of the Square:  25
Area of the Rectangle:  2000
Area of the Circle 81.0
PS C:\Users\Ganesh\Documents\VSC Python Projects\poly>
```

RUN TIME POLYMORPHISM

- Same method (single method) performs different operations using same set of signatures
- Signatures ← Number of arguments, Type of arguments
- This is called using its signatures
- Ex.

    1. Method Overriding

Existing Problem

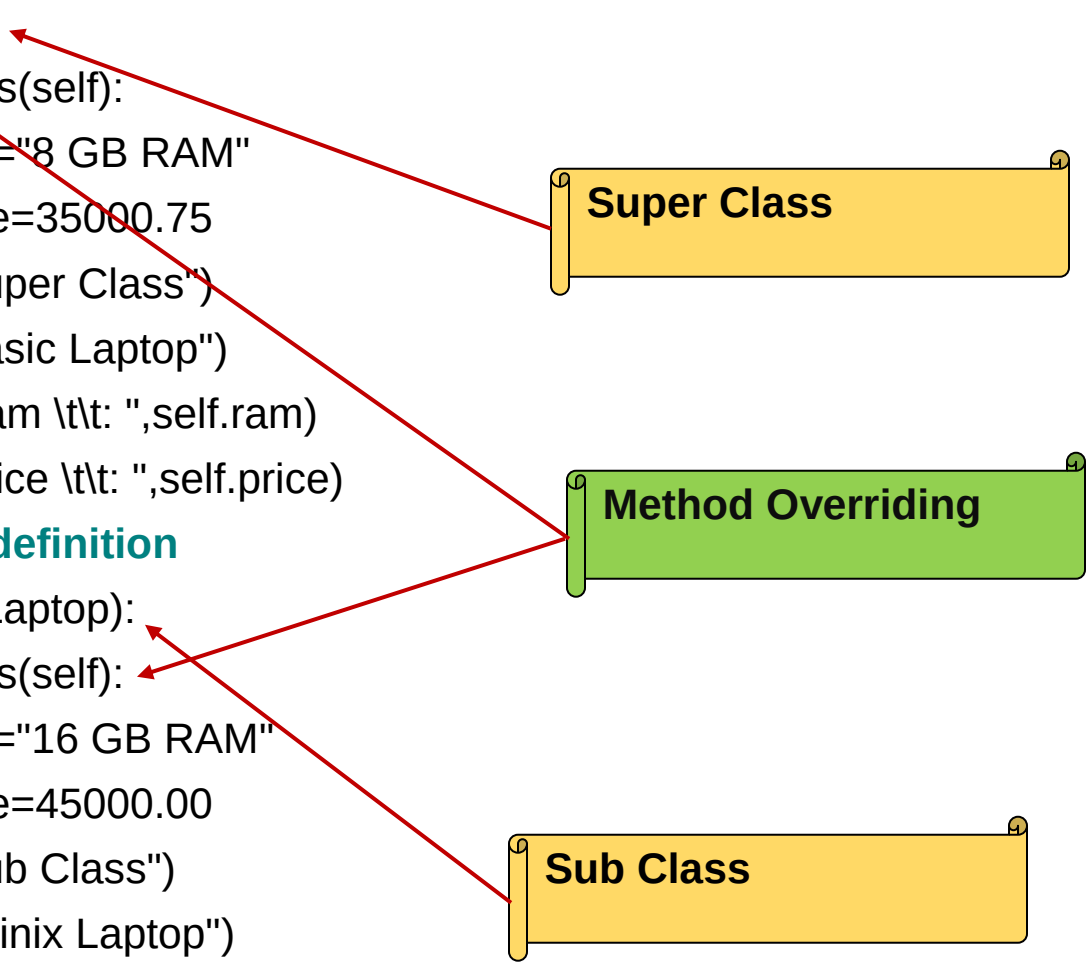
- If variables or methods or combination of both are **defined with same name in both super class** and **sub class**, then compiler / interpreter may confuse to call the methods. (This situation is called as ambiguity error)
- So only the methods of derived class will be executed forever and **super class methods will be hidden**.

2. ISSUES OF METHOD OVERRIDING

(overriding_general.py)

I. SOURCE CODE

```
# super class definition
class Laptop:
    def features(self):
        self.ram="8 GB RAM"
        self.price=35000.75
        print("Super Class")
        print("Basic Laptop")
        print("Ram \t\t: ",self.ram)
        print("Price \t\t: ",self.price)
# sub class definition
class Infinix(Laptop):
    def features(self):
        self.ram="16 GB RAM"
        self.price=45000.00
        print("Sub Class")
        print("Infinix Laptop")
        print("Ram \t\t: ",self.ram)
        print("Price \t\t: ",self.price)
```



```
# derived class object creation
obj=Infinix()
print("----------------------------------------------------")
print("\t Method Overriding without Solution")
print("----------------------------------------------------")
# calling both super and sub classes using derived class object
obj.features()
obj.features()
```

II. OUTPUT

```
TERMINAL    JUPYTER    PROBLEMS    OUTPUT    DEBUG CONSOLE
PS C:\Users\Ganesh\Documents\VSC Python Projects\poly> py overriding_general.py
----------------------------------------------------
         Method Overriding without Solution
----------------------------------------------------
Sub Class
Infinix Laptop
Ram             :  16 GB RAM
Price           :  45000.0
Sub Class
Infinix Laptop
Ram             :  16 GB RAM
Price           :  45000.0
PS C:\Users\Ganesh\Documents\VSC Python Projects\poly>
```

NOTE

- In the example above, by default, derived class methods are only called and **base class methods are hidden**.
- So in order to solve the issues above, python provides the special function called **super()** which is used to call the base class methods.

SOLUTIONS FOR RUN TIME POLYMORPHISM

- In the run time polymorphism, **the sub class (derived class) always hides the base class functions / variables, because of same signatures**
- To overcome this problem, python gives one solution. Namely
    1. Method Overriding using super() function

Method Overriding using Super() Function

- The super() function is used to access a variable, method of the super class that has the same name in the sub class.
- The super() function can be placed at anywhere (first statement or last statement) in the method of sub class

3. SOLVING METHOD OVERRIDING USING SUPER() FUNCTION (overriding.py)

I. SOURCE CODE

```
# super class definition
class Laptop:
    def features(self):
        self.ram="8 GB RAM"
        self.price=35000.75
        print("Super Class")
        print("Basic Laptop")
        print("Ram \t\t: ",self.ram)
        print("Price \t\t: ",self.price)
# sub class definition
class Infinix(Laptop):
    def features(self):
        self.ram="16 GB RAM"
        self.price=45000.00
```

```
        print("Sub Class")
        print("Infinix Laptop")
        print("Ram \t\t: ",self.ram)
        print("Price \t\t: ",self.price)
        print("-----------------------------------------------------")
        super().features()
# derived class object creation
obj=Infinix()
print("-----------------------------------------------------")
print("\t Method Overriding with Solution")
print("-----------------------------------------------------")

# calling both super and sub classes using derived class object
obj.features()
```

Calling super class's method

II. OUTPUT

```
TERMINAL    JUPYTER    PROBLEMS    OUTPUT    DEBUG CONSOLE

PS C:\Users\Ganesh\Documents\VSC Python Projects\poly> py overriding.py
-----------------------------------------------------
         Method Overriding with Solution
-----------------------------------------------------
Sub Class
Infinix Laptop
Ram             :  16 GB RAM
Price           :  45000.0
-----------------------------------------------------
Super Class
Basic Laptop
Ram             :  8 GB RAM
Price           :  35000.75
PS C:\Users\Ganesh\Documents\VSC Python Projects\poly>
```

DIFFERENCE BETWEEN OVERLOADING AND OVERRIDING

S.N	Overloading	Overriding
1.	Same method with different operations using different signatures	Same method with different operations using same signatures
2.	Compile time polymorphism (static binding or early binding)	Runtime polymorphism (dynamic binding or late binding)
3.	It does not need inheritance	It needs inheritance to achieve overriding concept
4.	Calling is based on its signatures	This is implemented using super() function.